

- Ensembles: what are they?

- * Dictionary definition of ensembles is, a group of musicians performing together.

- * In ML context, this is basically, multiple models used together.

- * Let's say we have,

$\langle M_1, M_2, M_3, M_4 \dots M_k \rangle$ $\xrightarrow{\text{combine}}$ More powerful models
Simple/Base models or More accurate models.

The models which we combine, could be either KNN, Logistic Regression, SVM, etc. We call them Simple/Base models

- * Basically, we have four major types of ~~models~~ ensembles:

1. Bagging (derived abbr. from Bootstrapped Aggregation)

2. Boosting

3. Stacking

4. Cascading

These are not just the only ensembles but more also exist.

Key Aspect: The more different the models are, the better you can combine them.

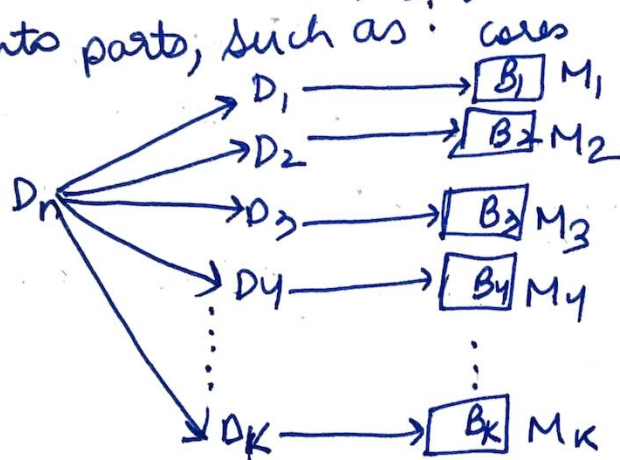
• Bagging Complexity (Random Forest)

* Let's say there are k -base learners which are fully grown decision trees, then,

$$\text{Train time complexity} = \underbrace{O(n \cdot \lg n \cdot d)}_{\text{complexity of DT}} \cdot \underbrace{k}_{\substack{\text{no. of} \\ \text{base} \\ \text{learners} \\ \text{which} \\ \text{is DT.}}}$$

* Here, training happens parallelly.

If training on single box, it will take above time. But, if you have divided dataset into parts, such as:



* So, the algorithm is trivially parallelizable because we generate the above bootstrap samples and train on different cores of boxes. So, when we have large amts. of data with reasonable no. of features or d (not too large), you can train thousands of trees. If you have Hadoop or Spark cluster, then it is too fast.

* Parallel algorithms are still under research in ML.

* Runtime Complexity : $O(d)$ $\because$ you need to traverse through the depth of DT to decide class label at leaf node.

* Runtime complexity for Random Forest having k -base learners is $\therefore O(d * k)$. Here, depth will also be large.

Typically around 10 to 20 not 2 or 3.

* k will depend on cross validation based on plot.

Space Complexity = (no. of leaf nodes + no. of internal nodes) $* k$.

as it depends on storage of nested if-else conditions.

Reasonable depth of tree can be around 6.

- Boosting Intuition

Let's understand Bagging first :-

In Bagging, we use High Variance and low bias models as our base learners. On top of it, we apply randomization. Randomization is done either by column sampling or row sampling. Also, we sometimes apply, random threshold (done in case of extremely randomized trees). Again, on top of randomization, we also apply aggregation like majority voting. Both randomization and aggregation is done to reduce high variance while keeping the bias (training error) low.

Boosting, on the other hand is ~~used~~ ~~not~~ done on models with base learners having low variance and high bias. On top of this, we apply additive methods to combine the base learners so as to reduce the bias (training error) while our variance is low.

There are many techniques of boosting but we'll focus on Gradient Boosting.

	Bagging (Base learners)	Boosting (Base learners)
Bias	Low	High
Variance	High	Low

Stage 0

- Core idea behind Boosting is to reduce high bias/high training error.

Let's say in step 0 ~~our~~ our whole train dataset is used for training the initial model M_0 .

Let's say this model M_0 is of a decision tree which is shallow. This model needs to have high bias and low variance. The model M_0 is trained by fitting the function $h_0(x)$. So, $y_i = h_0(x)$ will be the output from model M_0 .

$$\textcircled{a} \quad D_{\text{train}} \xrightarrow{\quad} M_0$$

$\{x_i, y_i\}_{i=1}^n$

Now, we'll subtract y_i and $h_0(x_i)$, so we get error as below:-

$$\textcircled{b} \quad \text{error}_i = y_i - h_0(x_i)$$

This error is definitely not the squared error. There are many types of errors, such as

- Squared loss/error
- logistic loss/error (In case of classification)
- Hinge loss/error

The error here is simply the difference b/w $y_i - h_0(x_i)$ and this error is non-trivial or not very simple or is very large.

~~The error term is bias~~

Each point in the dataset will have ~~the~~
 $\{x_i, y_i, \text{error}_i\}$

Stage 1

Since step 0 has trained the model on D_{train} this time we'll take $\{x_i, \text{error}_i\}_{i=1}^n$. So, this means we're training on errors in case of M_1 after stage 0.

$$\textcircled{a} \quad M_1 \leftarrow \{x_i, \text{error}_i\}_{i=1}^n$$
$$h_1(x_i) \approx \text{error}_i = y_i - h_0(x_i)$$

Model after stage 1:-

$$\textcircled{b} \quad F_1(x) = \alpha_0 h_0(x) + \alpha_1 h_1(x)$$

We call the above as weighted sum of two base models.

Stage 2

Now, again the model M_2 will be using $\{x_i, \text{error}_i\}_{i=1}^n$. This time, we'll combine the α_1 & α_2 and use $F_1(x_i)$ and subtract it from y_i , to get error_i:

$$\textcircled{a} \quad \text{error}_i = y_i - F_1(x_i)$$

After stage 2, we have:-

$$\textcircled{b} \quad F_2(x) = \alpha_0 h_0(x) + \alpha_1 h_1(x) + \alpha_2 h_2(x)$$

Stage k

At the end of step k, we get the final model. Here each base model $h_i(x)$ is trained to fit the residual error at the end of the previous stage which is $(i-1)$.

$$F_k(x) = \sum_{i=0}^k \alpha_i h_i(x)$$

The above is the additive weighted model.

Residual Error means = error which is left out,

Every model is trained on $h_i(x)$ which is $[x_i, \text{error}_i]$ which is residual error at the end of stage $i-1$.

$$\begin{array}{l} \text{error}_i \\ \text{(Residual Error)} \end{array} = y_i - F_{i-1}(x)$$

In an example, let's take $K=10$, so the model The trained ends up having a low residual error, so train error reduces.

As i increases, training error reduces and hence our bias also reduces by training on error.

Most popular algorithms based on boosting are Gradient Boosted Decision Trees (GBDT) and Adaptive Boosting.

Adaptive Boosting's most popular application is for Face Recognition.

$$F(x) = \sum_{i=1}^K \alpha_i h_i(x)$$

- Boosting complexity (GBDT)

* Train time complexity = $O(\underbrace{n \cdot \lg n \cdot d}_{\text{Decision Tree}})^* \underbrace{M}_{\substack{\downarrow \\ \text{No. of DT} \\ \text{or} \\ \text{No. of Base Learners}}}$

* Unlike Random Forest, GBDTs are ~~for~~ not easy to parallelize $\because$ it is a very sequential algorithm.

* As we see below,

Dataset 0

↓
Stage 0 Training → M_0

↓
Dataset 1

↓
Stage 1 Training → M_1

↓
Dataset 2

↓
Stage 2 Training → M_2

* So, you can't do stage 2 unless you have dataset 2 after stage 1. So, it is a very sequential step by step process.

* Hence, GBDTs are not trivial/easy to parallelize. GBDTs take more time to train than Random Forest. So, time complexity is almost same but still due to its sequential flow it lags.

* Runtime complexity = $O(d) * M$

Here, d = depth (small)

Runtime evaluation is much fast.

Since depth $> 1 \therefore \sim O(d) * M$.

Space complexity $\Rightarrow$ You have to store each of your trees.

* No. of nodes in total for + γ_m
each node

This is
for each base learner
(DT)

* So, in low latency applications, we can use GBDTs.

Property: You can use any loss function.

• Ensembles - Bias Variance Tradeoff

We see that bagging or random forest is a great way to reduce variance. Random forest have low bias and hence we use ~~them~~ as decision tree as our base learners in the ensemble technique. The M_i (Base Learners) are low bias to start with.

As far as variance is concerned our final model is M which is just the aggregation result of all the base learners.

$$M = \text{aggregation}(M_1, M_2, M_3, \dots, M_K)$$

(Final Model)

As K increases, which is nothing but our number of base learners, our variance decreases. which means the model's overfitting is less.

And as K decreases, the variance increases. The bias of the model is same as bias of individual models M_i . People train as max. as $2K$ base learners and aggregate them. So, K is one of the hyperparameters. Another hyperparameters are row sampling rate and column sampling rate. Since, we are training on D_{train} , we perform column sampling on 50% of the total train columns and row sampling on let's say 30% of the total rows.

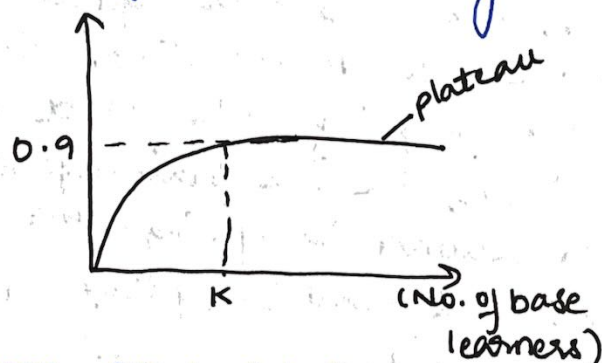
$$D_n \xrightarrow{\text{Row sampling}} D' \xrightarrow{\text{Column Sampling}}$$

Let's say d' is the new no. of columns in D' and n' is the ~~old~~ new no. of rows in D' . Let's suppose d is the old no. of columns in D_{train} and n is the old no. of rows in D_{train} .

We are also assuming that $d' < d$ and $m < n$. So, the column sampling ratio = $\frac{d'}{d}$ and similarly $\frac{m}{n}$ is the row sampling ratio. Then, how much of the total you're sampling determines the performance. As column sampling rate and row sampling rate should be low, then each of your datasets would be different and hence variance reduces. Don't make the sampling rate as low as 0.1%, otherwise, there would be no points in D_{train} to train upon.

NOTE: Your ~~to~~ cross-validate and Test datasets are untouched.

You have to always alter K (No. of Base Learners) and hence performance changes.



Let's assume accuracy is our metric, then as per graph as K increase performance increases and then it plateaus.

Hyperparameters

1. Column Sampling Rate
 2. Row Sampling Rate
 3. No. of Base Learners (K)
- ↑ Most imp.

As far as Bias-Variance is concerned, we play with K and use DT $\therefore$ it has low bias & high variance and reduce it in training.

V2: Bootstrapped Aggregation (Bagging) Intuition

Bagging
|
Bootstrapped

$$m < n$$

$D_{\text{Train}}(n \text{ points})$
 (x_i, y_i)

Sampling with
replacement
mpts

$D_m^1 \xrightarrow{\text{Train}} M_1$

Sampling with
replacement

$D_m^2 \xrightarrow{\text{Train}} M_2 \rightarrow M$

Sampling with
replacement
 $D_m^k \xrightarrow{\text{Train}} M_k$
(Bootstrap sample)

Agg
reg
ation

Aggregation $\begin{cases} \text{Classification} \rightarrow \text{Majority Vote} \\ \text{Regression} \rightarrow \text{Mean/Median} \end{cases}$

Variance $\rightarrow$ degree of change in model with change in data

Say we changed 100 pts of 10k points. Only a subset of the datasets would get affected, that also to a very less extent. So, overall model doesn't change much.

Bagging $\rightarrow$ Can reduce variance in a model without impacting the bias.

$$\text{Model error} = \text{Bias}^2 + \text{Variance}$$

If the base models have low bias + high variance, then bagging is just the perfect ensemble.

Bagging on M_i 's $\rightarrow$ low bias, reduced variance

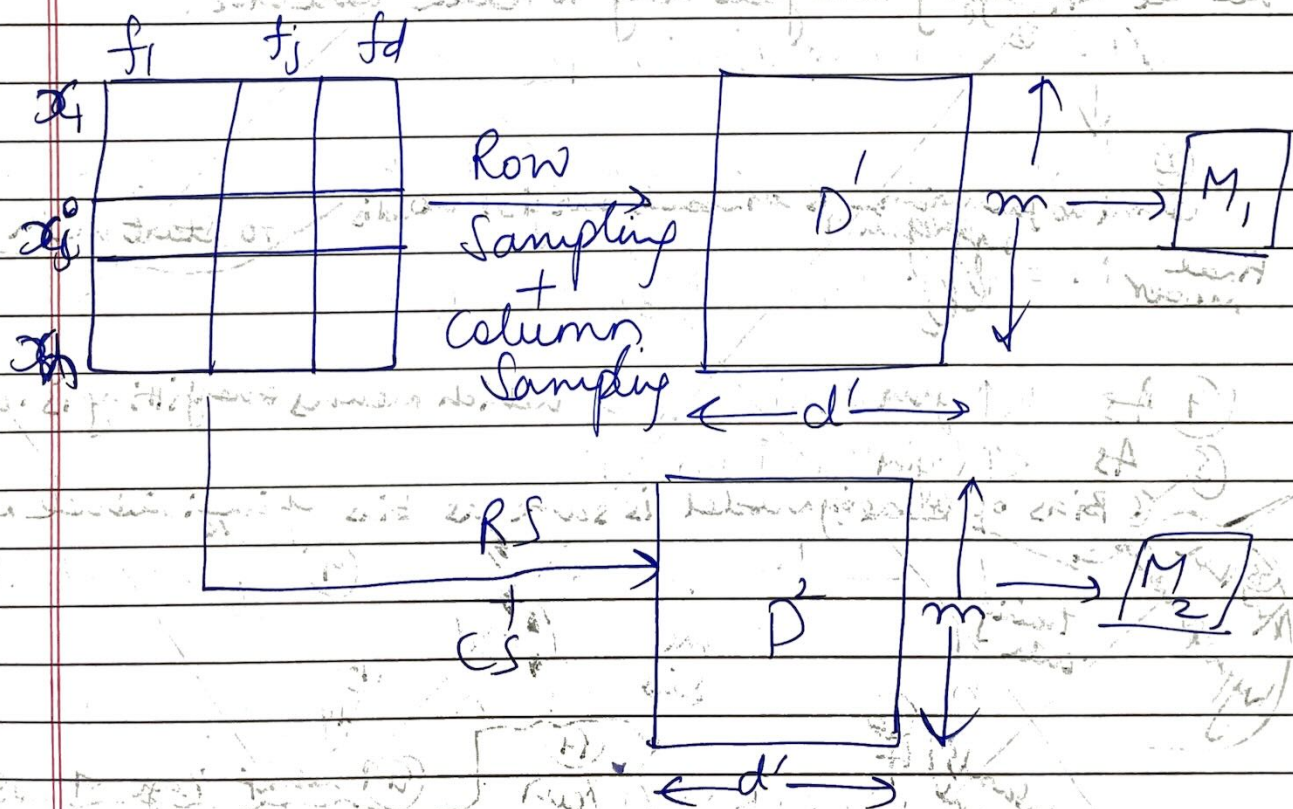
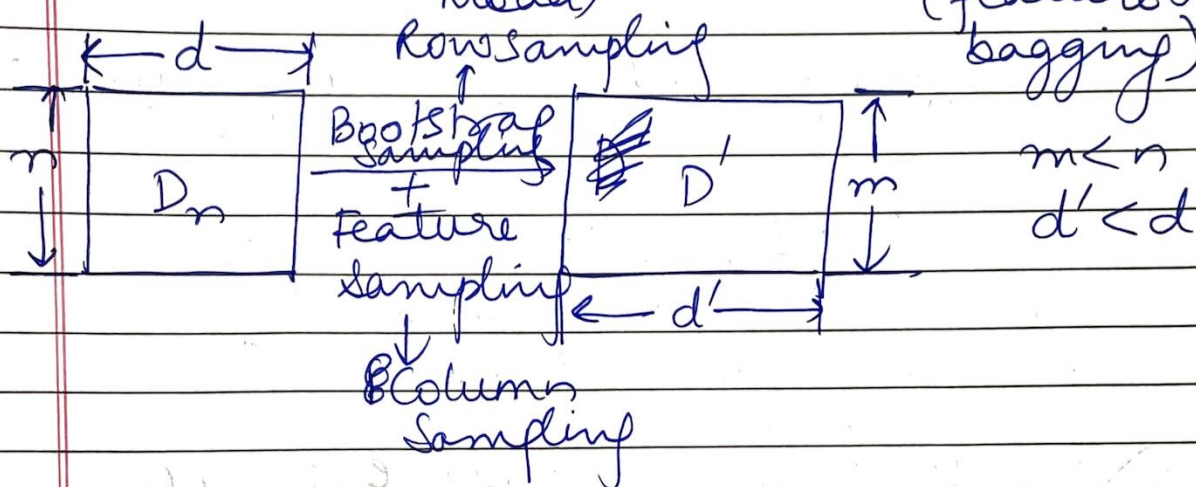
eg- Decision Tree of good depth $\rightarrow$ low bias and high variance

Bagging DT $\rightarrow$ Random Forests.

V3: Random Forest and their construction

Random - Random Bootstrap Sampling.
Forests - Collection of decision trees.

Random Forest $\rightarrow$ DT + Bagging + column Sampling (feature bagging)



(16) People say we sample 50% of cols & 80% of rows.



Sampled datasets now have different sets of features each.

M_i 's $\rightarrow$ Decision Trees of reasonable depth
(low bias + high variance)

Model M_i uses D_i for training.

So, $\underbrace{D_n - D_i}_{\substack{\text{out of Bag (OOB) samples} \\ \text{OOB error is also returned.}}}$ used as cross validation set.

RF $\rightarrow$ very powerful.

$h_0(x_i) \rightarrow M_0$ is fitting function

V7: Extremely Randomized Tree

Only change in Random Forest is while handling real value features, instead of trying each & every value after sorting for threshold τ , try from a random sample of numbers to make a bit faster & introduce some randomization.

Column Sampling + Row Sampling + Randomization when selecting τ $\rightarrow$ RF

V8: Except Bias-Variance Tradeoff and Feature Importance, Decision Tree and Random Forest work same.

V10 Residuals, Loss Functions and Gradients

Residual

at end of stage $k \Rightarrow e_i = y_i - F_k(x_i)$

Train model M_{k+1} by fitting its function to $\{x_i, e_i\}_{i=1}^n$

~~Assuming regression~~, we define loss at end of stage k as:-

Squared loss - $L(y_i, F_k(x_i)) = (y_i - F_k(x_i))^2$ True class label Model @ end of stage k

derivative $\frac{\partial L}{\partial F_k(x_i)} = 2(y_i - F_k(x_i)) * (-1)$

$\Rightarrow \frac{-\partial L}{\partial F_k(x_i)} = 2(y_i - F_k(x_i))$

negative derivative of loss wrt model at end of stage k error

residual

Negative derivative $\approx$ Pseudo residual

Gradient boosting utilizes this.

We can train on pseudo-residuals.
We can train on any loss function which is differentiable.

~~Random forest min. hinge loss~~

So, what gradient boosting does is:-

$$\begin{matrix} D \\ \{x_i, e_i\}_{i=1}^n \end{matrix} \longrightarrow \begin{matrix} M \\ e_i = h_k(x_i) \end{matrix}$$

where

$$e_i = \text{pseudo residual, ie, } -\frac{\partial L}{\partial F_{k-1}(x)}$$

This allows us to use any differentiable loss function.

~~So we can minimize hinge loss using decision trees.~~

* So Gradient Boosted Decision Trees (GBDT) tends to perform better than random forests... of pseudoresiduals.

VIII Gradient Boosting
Input: Training Set $\{(x_i, y_i)\}_{i=1}^n$, a differentiable

loss function $L(y, F(x))$, number of iterations M (no. of base models)

~~VIII Gradient Boosting~~
Algorithm -

1. Initialize model with constant value

$$F_0(x) = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, \gamma) \quad \text{if } L \text{ is sq. loss } \gamma = \operatorname{mean}(y_i)$$

gamma

2. For $m=1$ to M :

a. Compute the pseudo residuals

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x) = f_{m-1}(x)}$$

$i = 1, 2, \dots, n$.

b. ~~Form~~ Fit a base learner (eg. tree) $h_m(x)$ to pseudo residuals, ie, train it using the training set $\{(x_i, r_{im})\}_{i=1}^n$

c. Compute multiplier γ_m by solving the following 1-dimensional optimization problem.

optimization equation

$$\gamma_m = \underset{\gamma}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i))$$

d. Update the model

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x).$$

3. Output $F_m(x)$.

~~~~~

## V12: Regularization by Shrinkage

At the end of the algorithm, we get:-

$$F_M(x) = h_0(x) + \sum_{m=1}^M \gamma_m h_m(x)$$

$M = \#$  of base models (hyperparameter)

$M \uparrow \Rightarrow \text{overfit} \uparrow \Rightarrow \text{Variance} \uparrow$

Optimal 'M' could be found by cross validation and GridSearch or Random Search.

### Alternatively Shrinkage

$$F_M(x) = F_{M-1}(x) + \underbrace{\gamma}_{\text{learning rate}} h_m(x)$$

$$[0 < \gamma \leq 1]$$

$\gamma \uparrow \Rightarrow \text{overfit} \uparrow \Rightarrow \text{Variance} \uparrow$

$\gamma \downarrow \Rightarrow \text{Variance} \downarrow$

So, now we have 2 hyperparameters:  $M$  &  $\gamma$   
 $\gamma = \text{Shrinkage Coefficient}$

CV with GridSearch or RandomSearch ( $M, \gamma$ )



## V14 XGBoost : Boosting + Randomization

Gradient Boosting (but not sklearn).

XGBoost : GBDT +  $\begin{pmatrix} RS \\ CS \end{pmatrix}$  ~~SKLE~~RF

$\eta$  = learning rate

~~M~~ =

$M$  = n-estimators.

$$LR = w_0 + w_1 x_{i1} + w_2 x_{i2} + \dots$$
$$GB = h_0(x) + \gamma_1 h_1(x) + \gamma_2 h_2(x) + \dots$$

Common thing b/w Logistic Regression and Gradient Boost  $w_i$  and  $\gamma_i$  is weight.

Two points -



Output: An ensemble Classifier  $H$ .

1. Learn First-level classifiers (base model)  
 for  $i=1$  to  $T$  do  
     Learn a base classifier  $h_i$  based on  $D_0$   
 end for

2. Construct new dataset from  $D$  using trained models.  
 for  $i=1$  to  $m$  do  
     Construct a new dataset that contains  $\{x'_i, y_i\}$   
     where  $x'_i = \langle h_1(x_i), h_2(x_i), h_3(x_i), \dots, h_T(x_i) \rangle$   
 end for

3. learn a second level classifier (Meta classifier)  
 learn a new classifier  $h'$  based on the newly constructed dataset  
 return  $H(x) = h'(h_1(x), h_2(x), h_3(x), \dots, h_T(x))$

\*  $h_i(x)$  - can be  $\begin{cases} \text{o/p label} \\ \text{probability value} \end{cases}$

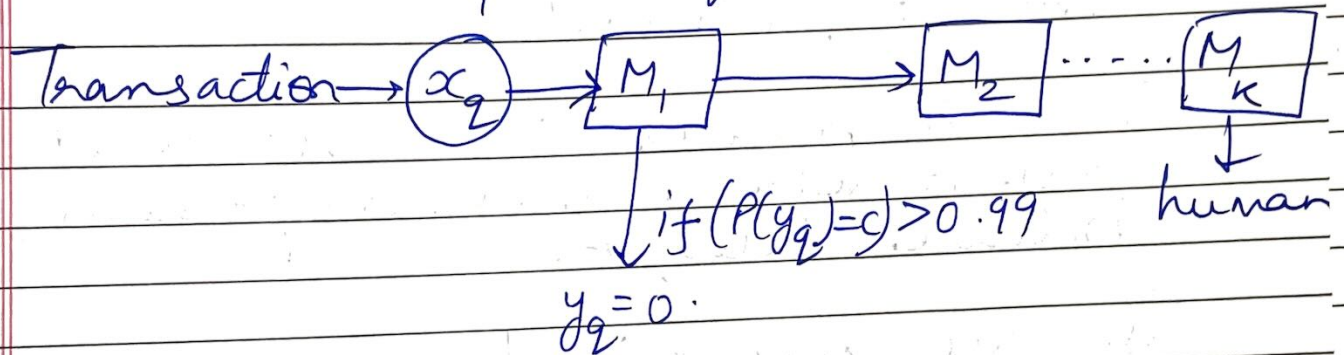
Kaggle uses stacking exclusively.



## V17: Cascading Models

eg- System that predicts credit card transaction is fraud or not.

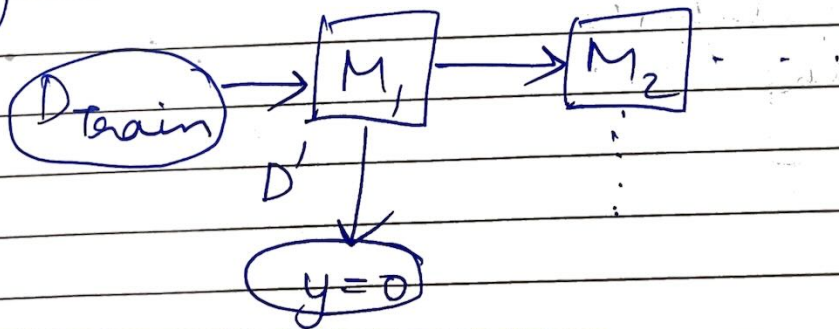
Core idea: If very sure that its negative  
↓  
op as negative.  
else pass on for further processing.)



After processing through all the cascaded model, if still can't be decided, human intervention required.

These models are used where the cost of False Negatives is very high.

eg - Cancer detection.



$$D_{train}^2 = D_{train} - D' \quad \text{and so on.}$$

## V18: Kaggle vs Real World

1) Kaggle cares about a single metric—

logloss

F1-Score

AUC

Real World—multiple metrics with different priorities.

This is because in real world we try to solve a business problem, i.e., increase sales, etc.

So, we map this business metric  $\longrightarrow$  multiple ML metrics

2) Kaggle—very complex ensembles because goal to improve 1 metric.

This might be impractical in real world due to:

- i) low latency
- ii) interpretability
- iii) training time